

0.107.2. Tuile — dessiner une case

#include <Tuile> : Tuile NOM = { huit rangées de huit pixels }; dessine une case à toi, que poser() pose comme une lettre.

1. Ce qu'il faut comprendre

Ce que c'est : une **Tuile** est le dessin d'une case : 8 rangées de 8 pixels. Chaque pixel s'écrit en **chiffre** (0 le plus clair, 3 le plus foncé) ou en **signe** (., -, +, #).

La ligne à écrire : **#include <Tuile>**. Un dessin écrit sur place, dans un `poser(..., { ... })`, la demande aussi.

Ce qu'elle coûte : **16 octets** par tuile : deux octets par rangée.

Essaie : change une rangée en `"#####"` : une barre apparaît.

On retrouve Tuile dans 56 leçons

[0.108](#), [3](#), [34](#), [35](#), [36](#), [37](#), [38](#), [40](#), [41](#), [42](#), [43](#), [45](#), [46](#), [48](#), [49](#), [50](#), [51](#), [52](#), [61](#), [72](#), [73](#), [74](#), [83](#), [84](#), [85](#), [95](#), [96](#), [97](#), [98](#), [99](#), [100](#), [101](#), [102](#), [103](#), [104](#), [105](#), [106](#), [107](#), [108](#), [109](#), [110](#), [111](#), [112](#), [113](#), [122](#), [124](#), [125](#), [127](#), [128](#), [131](#), [132](#), [134](#), [136](#), [137](#), [140](#), [143](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **181 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         19 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
CopierTuiles   31 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles – 0 lettre sur 43, et 1 tuile dessinée
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 2    12 o sur place, dans main()
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

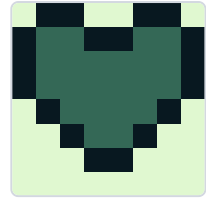
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <Tuile> : dessiner une case ----
// Un cœur : # le plus foncé, + moyen, . vide.

#include <Tuile> // Tuile : un dessin de 8 × 8 pixels
#include <poser>
```

2. UN DESSIN — COEUR

```
Tuile COEUR = {  
    ".##..##.",  
    "#####",  
    "#####",  
    "#####",  
    "#####",  
    ".#####",  
    "..###..",  
    "...##...",  
    ".....",  
};
```



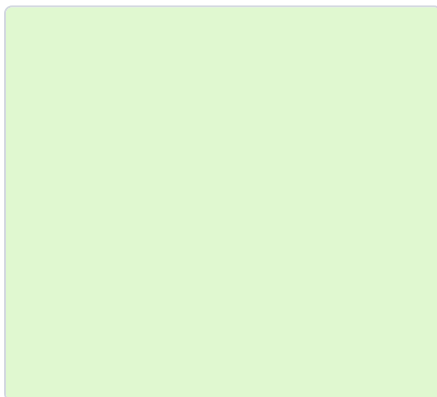
à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

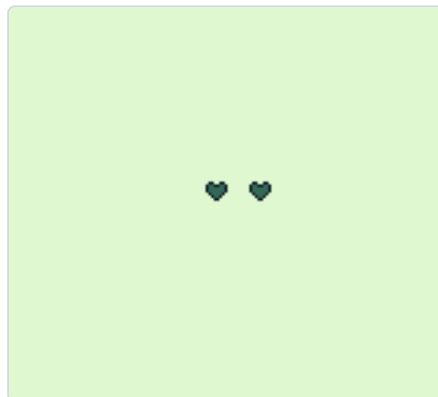
```
int main() {  
    poser(9, 8, COEUR);    // le nom de la tuile se pose comme une lettre  
    poser(11, 8, COEUR);   // deux fois : la tuile n'est gravée qu'une fois  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux cœurs au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ un cœur en (9, 8)

✓ le même en (11, 8) : une seule tuile

Ensuite : **0.107.3. La fonction `cache()` — ôter un lutin** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++